

→
Курси

Master
of
Science:
Data
Science
and
Data
Analytics

→
М.. ▾

Пошук



Міні-курси



 NEW

✓ Огляд дисципліни

✓ Тема 1. Вступ до MLOps

✓ Наукова публікація 1

✓ Тема 2. Основи Linux і Bash-скриптування

✓ **Тема 3. Контейнеризація ML-моделей**

📖 Навчання

✓ Вступ

✓ Використання Docker для ML

✓ Створення Docker-імеджу для ML-моделей

✓ Оптимізація Docker-image для ML-моделей

✓ **Оркестрація контейнерів за допомогою Docker Compose**

— Підсумки

📁 Матеріали

📺 Уроки

💻 Домашнє завдання

✓ Тема 4. Вступ до IaC в контексті MLOps та Kubernetes

✓ Тема 5. Kubernetes для MLOps

✓ Тема 6. Деплой ML-сервісів у Kubernetes (EKS)

✓ Тема 7. ArgoCD для Helm-деплою

✓ Тема 8. Моніторинг та логування моделей

✓ Тема 9. Моніторинг якості моделей та відстеження експериментів

✓ Тема 10. Автоматизоване тренування моделей

✓ Тема 11. Управління моделями (Model Registry & Deployment Strategies)

- ✓ Тема 12. Безпека ML-моделей: принципи, загрози та захист
- ✓ Наукова публікація 2
- ✓ Фінальне тестування

Оркестрація контейнерів за допомогою Docker Compose

Як керувати моделлю, API та залежностями в одному середовищі?

У попередніх підтемах ми запакували ML-модель у Docker-контейнер. Тепер її можна запускати будь-де однією командою, наприклад `docker run`.

Але в реальному житті **ML-модель майже ніколи не працює сама по собі**. Вона зазвичай є частиною більшого застосунку, наприклад:

- Вебсервісу, який приймає HTTP-запити (FastAPI, Flask, Django);
- Кешу або черги (Redis, RabbitMQ), що передає дані;
- Бази даних для логування запитів або збереження результатів.

Усі ці частини повинні **працювати разом**, мати **спільну мережу**, обмінюватись даними — інакше не буде сенсу запускати модель.

Запускати їх вручну по черзі довго, незручно, і легко помилитись.

Саме тому тут з'являється поняття **оркестрації**, а для локальної розробки й невеликих проєктів **ідеальний варіант — це Docker Compose**.

Що таке Docker Compose?

! Docker Compose — це інструмент, який дозволяє керувати кількома пов'язаними контейнерами **за допомогою одного YAML-файлу**.

Це як **інструкція для оркестратора**: у файлі `docker-compose.yml` ви описуєте, які сервіси треба запустити, які образи вони використовують, які порти відкривають, як з'єднані між собою, які мають змінні середовища або залежності.

Після цього — лише одна команда:

```
docker-compose up
```

І весь «ML-сервіс» запускається **одразу**: модель, FastAPI-сервер, Redis, база та інші залежності. 🎉

Навіщо це потрібно для ML?

Коли ви деплоїте **модель як HTTP-сервіс**, ви зазвичай хочете:

- Приймати зображення або текст через API (FastAPI);
- Можливо, ставити запити в чергу (через Redis);
- Логувати метадані запитів (у базу);
- Масштабувати модель за запитом (наприклад, декілька екземплярів inference-сервісу).

Це вже мінімум **2–3 окремі контейнери**, які потрібно не просто запустити, а **синхронно оркеструвати** як частину ML-системи.

У контексті **MLOps** ваша задача — не писати код для моделі, а **забезпечити її стабільне, кероване, автоматизоване розгортання**.

І ось тут Docker Compose — це **локальний інструмент оркестрації**, який дозволяє:

- Перевірити, як модель працює як частина сервісу;
- Підняти всю інфраструктуру моделі (FastAPI, Redis, база тощо) у вигляді мікросервісів;
- Протестувати залежності перед деплоєм у Kubernetes чи хмару.

Загальна структура проєкту:

```
**pytorch-image-classifier/
├── app/
│   ├── inference.py      # старий CLI-сценарій
│   ├── main.py           # FastAPI-сервер
│   ├── model_utils.py    # логіка для моделі (спільна)
│   └── requirements.txt  # усі залежності
├── model/
│   └── traced_model.pt   # TorchScript-модель
├── example.jpg           # тестове зображення
├── Dockerfile            # один універсальний образ
├── docker-compose.yml    # сервіс FastAPI
└── save_model.py         # скрипт для генерації .pt**
```

КРОК 1. app/model_utils.py — винесення логіки інференсу

```
import torch
from torchvision import transforms
from PIL import Image
import io
```

```

# Load the TorchScript model
model = torch.jit.load("model/traced_model.pt")
model.eval()

# Define preprocessing pipeline
transform = transforms.Compose([
    transforms.Resize(256),
    transforms.CenterCrop(224),
    transforms.ToTensor()
])

# Predict from raw bytes
def predict_image(image_bytes: bytes) -> int:
    image = Image.open(io.BytesIO(image_bytes)).convert("RGB")
    tensor = transform(image).unsqueeze(0)
    with torch.no_grad():
        output = model(tensor)
        return output.argmax(1).item()

```

КРОК 2. app/main.py — FastAPI-сервер

```

from fastapi import FastAPI, File, UploadFile
from app.model_utils import predict_image

app = FastAPI()

@app.get("/")
def read_root():
    return {"message": "ML inference API is ready 🚀"}

@app.post("/predict")
async def predict(file: UploadFile = File(...)):
    image_bytes = await file.read()
    class_id = predict_image(image_bytes)
    return {"predicted_class": class_id}

```

КРОК 3. app/inference.py

```

import sys
from app.model_utils import predict_image

if __name__ == "__main__":
    with open(sys.argv[1], "rb") as f:

```

```
result = predict_image(f.read())  
print(f"🧠 Predicted class: {result}")
```

КРОК 4. app/requirements.txt

```
torch  
torchvision  
pillow  
fastapi  
uvicorn[standard]  
python-multipart
```

КРОК 5. Dockerfile

```
FROM python:3.11-slim  
  
WORKDIR /app  
  
# Copy model, code, requirements  
COPY model/ /app/model/  
COPY app/ /app/app/  
COPY app/requirements.txt .  
  
# Install dependencies  
RUN pip install --no-cache-dir -r requirements.txt  
  
# Default command: run FastAPI server  
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port",  
"8000"]
```

КРОК 6. docker-compose.yml

```
version: "3.9"  
  
services:  
  model-api:  
    build: .  
    ports:  
      - "8000:8000"  
    volumes:  
      - ./model:/app/model
```

КРОК 7. Запуск

```
docker compose up --build
```

Далі

